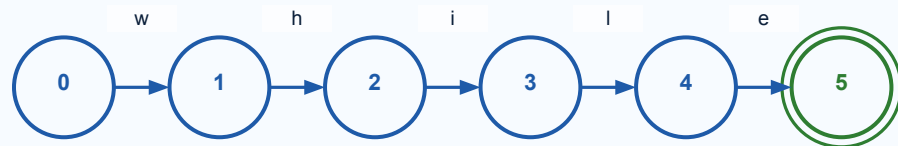


Lex / Flex

Lexical Analysis and Parser Interface

UG CSE lecture deck • inspired by Aho–Lam–Sethi–Ullman compiler principles



```
{ return TOKEN; }
```

Learning Outcomes

By the end, students should be able to build a small scanner and connect it to a parser.

Theory

Explain token, lexeme, pattern and attribute.
Convert regular expressions into DFA intuition.
Write flex rules for keywords, identifiers, numbers and comments.

Practice

Use longest match and rule priority correctly.
Return tokens and semantic values to yacc/bison.
Debug common scanner mistakes.

Where Lexical Analysis Fits

The scanner is the compiler front-end stage closest to source characters.



Scanner output is a token stream: <token-name, optional-attribute>

Token, Lexeme, Pattern, Attribute

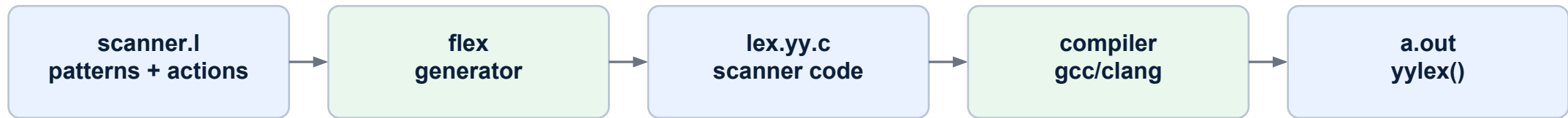
Four basic words used throughout lexical analysis.

Token	Abstract category returned to parser	ID, NUM, IF, PLUS
Lexeme	Actual character sequence in source	count, 42, if, +
Pattern	Rule that describes lexemes	[A-Za-z_][A-Za-z0-9_]*
Attribute	Extra value attached to token	symbol-table pointer, number value

In flex: pattern is on the left, action is on the right.

What Lex/Flex Generates

lex and flex generate C scanner code from regular-expression rules.



Key idea

You specify what tokens look like, not every character-by-character control path. flex builds a finite automaton internally and emits a fast scanner.

The Three Sections of a Flex File

A flex specification has declarations, rules and user code.

```
%{  
/* C declarations: headers, globals */  
%}  
  
definitions  
%%  
patterns      actions  
%%  
user C code
```

Meaning

Section 1: C declarations and named regex definitions

Section 2: regular-expression rules and actions

Section 3: helper functions, main(), yywrap()

Minimal Flex Program

A complete scanner can be written in a few lines.

```
%{
#include <stdio.h>
%}
%%
[0-9]+      { printf("NUM(%s)\n", yytext); }
[a-zA-Z]+   { printf("WORD(%s)\n", yytext); }
[
]+         { /* skip whitespace */ }
.           { printf("CHAR(%s)\n", yytext); }
%%
int main(void) { return yylex(); }
int yywrap(void) { return 1; }
```

Core runtime variables

yytext = matched lexeme
yyleng = lexeme length
yylex() = scanner function
yywrap() = input-end handler

Run flow

flex file.l → lex.yy.c → cc lex.yy.c -lfl → ./a.out

Regular Expressions in Flex

Flex regex syntax is close to compiler theory, with useful scanner shortcuts.

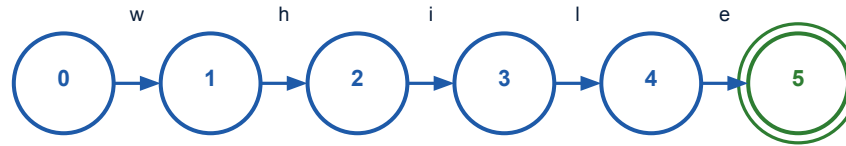
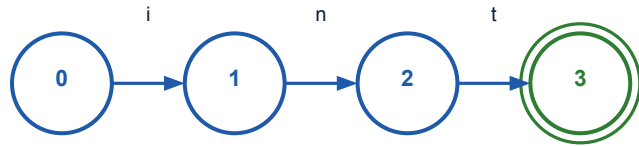
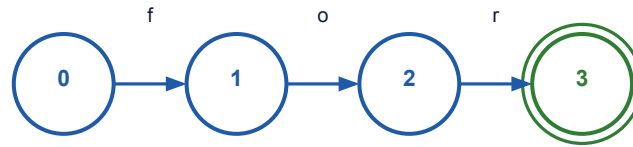
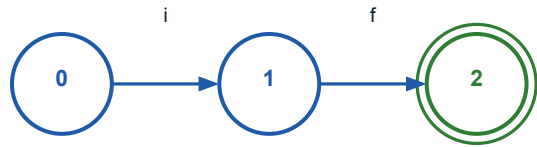
a	literal character a
[abc]	one of a, b, c
[a-z]	range
r*	zero or more
r+	one or more
r?	optional
r s	alternative
rs	concatenation
{name}	named definition
\n	newline escape

```
DIGIT  [0-9]
ID     [A-Za-z_][A-Za-z0-9_]*
INT    {DIGIT}+
FLOAT  {DIGIT}+"."{DIGIT}+
```

Named definitions make token patterns readable and reusable.

Finite Automata View

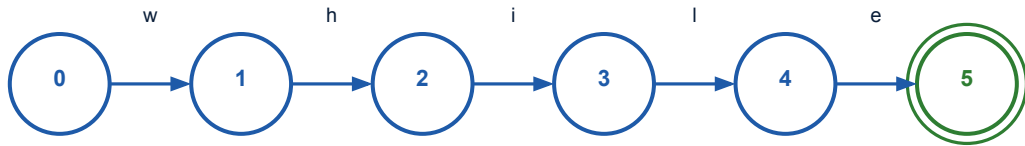
A flex scanner can be understood as many token automata combined into one DFA.



Each keyword can be represented as a small DFA path ending in a final state.

Keyword Automata: Direct Recognition

Scanner can recognize reserved words before classifying identifiers.



Important ordering

Put keyword rules before the general identifier rule. Otherwise "while" may be returned as ID.

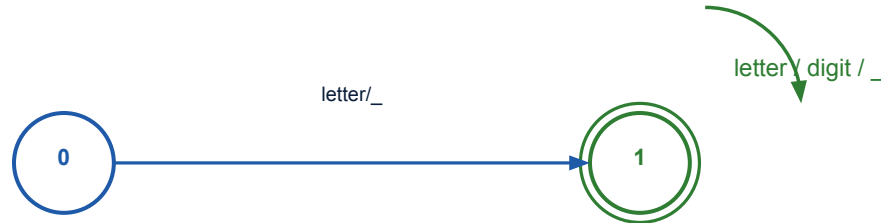
Compiler practice

Alternative: return ID for every name, then look up whether the lexeme is reserved in a keyword table.

```
"while"    return WHILE;
"if"       return IF;
"else"     return ELSE;
[A-Za-z_][A-Za-z0-9_]* return ID;
```

Identifier DFA

Identifiers are usually letter/underscore followed by letters, digits or underscores.



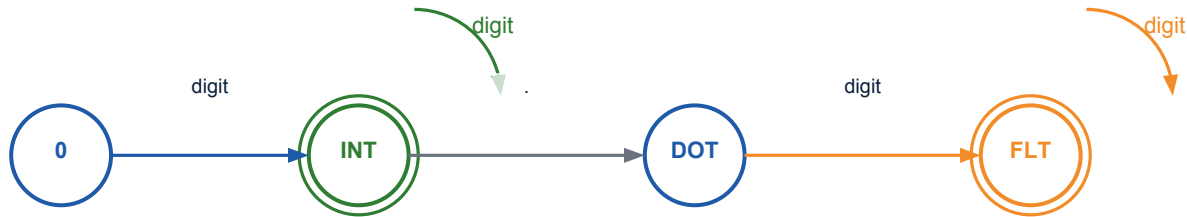
```
ID    [A-Za-z_][A-Za-z0-9_]*  
%%  
{ID} { return ID; }
```

DFA intuition

Once the first character is valid, the scanner stays in the final identifier state as long as valid identifier characters continue.

Number Automata: Integers and Floats

Numbers are a good example of prefix ambiguity.



Point to notice

12.34 should be one FLOAT token, not INT(12), DOT, INT(34).
Longest match solves this.

```
DIGIT  [0-9]
INT    {DIGIT}+
FLOAT  {DIGIT}+"."{DIGIT}+
```

```
{FLOAT} return FLOAT;
{INT}   return INT;
```

Longest Match Rule

Flex chooses the rule that matches the longest input prefix.

Input: while1

Rule candidates

"while" matches 5 characters
ID matches 6 characters



Flex result

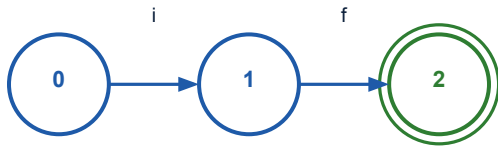
ID wins because it consumes longer text: while1

```
"while"    return WHILE;  
{ID}      return ID;
```

Tie-Break Rule Priority

If two rules match the same length, the earlier rule wins.

```
"if"    return IF;  
{ID}   return ID;
```



Input: if

Keyword rule and ID rule both match length 2. Since "if" appears first, token IF is returned.

Teaching warning

Put specific rules before general rules. This is the simplest way to avoid subtle scanner bugs.

Whitespace and Newline Handling

Scanners usually skip whitespace but track line numbers.

```
%option noyywrap
%{
int line_no = 1;
%}
%%
[
]+      ;      /* ignore blanks */

        { line_no++; }
.        { return yytext[0]; }
%%
```

Why track lines?

Line and column location make compiler errors useful: "syntax error near x at line 7".

Scanner policy

Whitespace is often not a token in programming languages, but it still affects diagnostics.

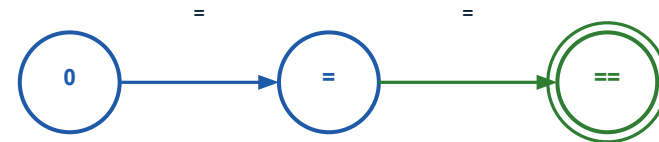
Operators and Delimiters

Many single-character tokens can return the character itself.

```
"=="      return EQ;  
"!="      return NE;  
"<="      return LE;  
">="      return GE;  
"="       return '=';  
"+"|"-"|"*"|"/" return yytext[0];  
"("|")"|"{"|"}" return yytext[0];
```

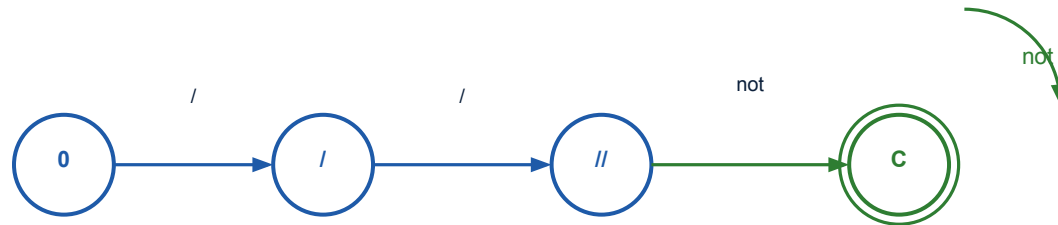
Longest match again

For input ==, flex chooses "==" over "=" because it is longer.



Comment Recognition: // Comments

Line comments are easy: read until newline.



Meaning

Match //, then zero or more characters except newline. The newline remains for the line-count rule.

```
"//"[^  
]*      ; /* skip line comment */
```

Comment Recognition: /* ... */

Block comments need care because regex greediness can over-consume.

```
"/*(\[^\*]|*\[^\*/\])**+/" ;
```

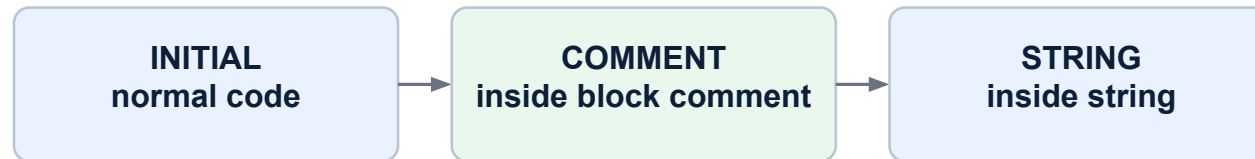
Practical flex approach

Use a start condition COMMENT and consume the comment in pieces. This is easier to debug and can count newlines.

```
%x COMMENT
%%
"/*"      BEGIN(COMMENT);
<COMMENT>"*/" BEGIN(INITIAL);
<COMMENT>
    line_no++;
<COMMENT>. ;
```

Start Conditions

Start conditions let the scanner switch mini-modes.



Inclusive vs exclusive

%s creates inclusive states; %x creates exclusive states. For teaching scanners, %x is often clearer.

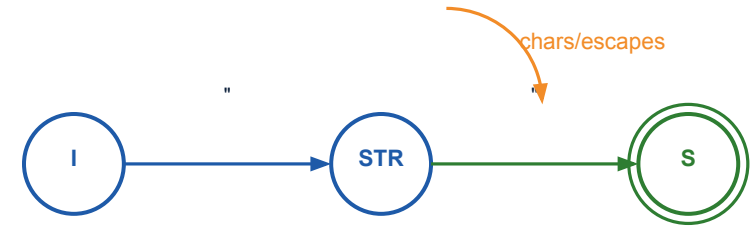
Common uses

Block comments, string literals, preprocessor lines, language embeddings and indentation-sensitive lexing.

String Literal Scanner

A string scanner must handle escapes and unterminated strings.

```
%x STR
%%
"      { BEGIN(STR); buf_clear(); }
<STR>\n { error("unterminated string"); }
<STR>\\. { buf_add_escape(yytext); }
<STR>"  { BEGIN(INITIAL); return STRING; }
<STR>[^\\
]+ { buf_add(yytext); }
```



Design rule

Do not try to handle every string case with one unreadable regex when a state-based scanner is clearer.

Echo, Reject and Default Rules

Flex has powerful features, but many compilers keep scanners simple.

Usually recommended

- Use explicit catch-all rule
- Track errors and locations
- Use named definitions
- Keep actions short

Use carefully

- ECHO prints yytext
- REJECT backtracks to other rules
- yyomore() appends next token text
- yyless(n) pushes part back

Error Token Handling

A robust scanner should fail gracefully on illegal characters.

```
. {  
    fprintf(stderr,  
        "Illegal character '%s' at line %d\n",  
        yytext, line_no);  
}
```

Policy choices

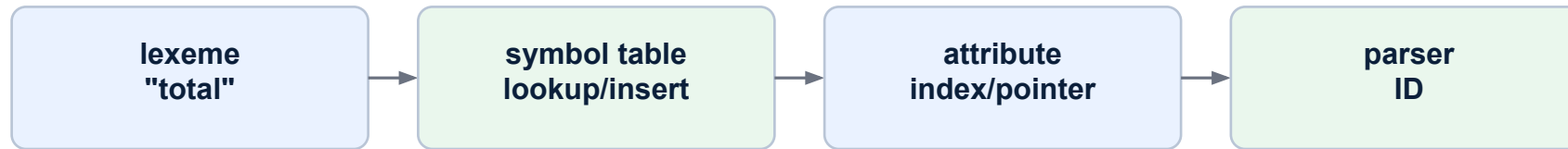
1. Report and continue
2. Return ERROR token to parser
3. Abort compilation after threshold

Classroom tip

Always include a final catch-all rule. Without it, flex may silently echo unmatched characters.

Symbol Table Interaction

Identifiers usually produce an ID token plus an attribute.

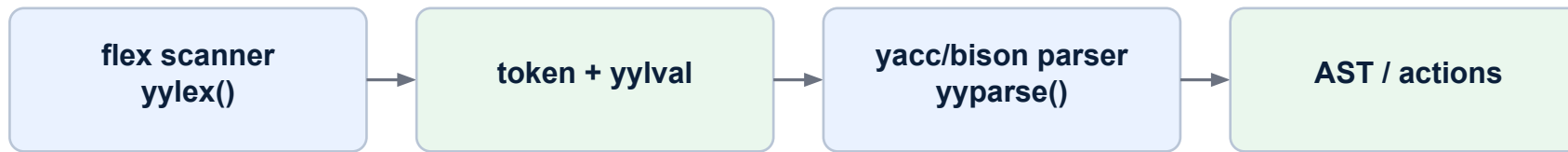


```
{ID} {  
    yylval.sym = sym_lookup(yytext);  
    return ID;  
}
```

Compiler idea

Lex and Yacc/Bison: Division of Work

Scanner recognizes words; parser recognizes sentence structure.



Scanner responsibility

- Characters → tokens
- Skip whitespace/comments
- Attach token attributes

Parser responsibility

- Tokens → grammar structure
- Detect syntax errors
- Build parse tree / AST

Yacc Token Declarations

The parser declares the symbolic tokens it expects.

```
%token ID NUM IF ELSE WHILE
%token EQ NE LE GE
%left '+' '-'
%left '*' '/'
%%
expr : expr '+' expr
     | expr '*' expr
     | '(' expr ')'
     | NUM
     | ID
     ;
```

Scanner must return

The exact token names declared by yacc/bison, or literal characters such as +, -, (,).

Practical note

bison -d parser.y creates parser.tab.h containing token definitions for flex.

Flex File Connected to Yacc

Include the generated parser header so token codes match.

```
%{
#include "parser.tab.h"
%}
DIGIT [0-9]
ID [A-Za-z_][A-Za-z0-9_]*
%%
"if"      return IF;
"while"   return WHILE;
{DIGIT}+  { yylval.ival = atoi(yytext); return NUM; }
{ID}      { yylval.sval = strdup(yytext); return ID; }
[
]+ ;
.         return yytext[0];
```

Why include parser.tab.h?

It defines IF, WHILE, NUM, ID etc. as integer token codes known to both scanner and parser.

yylval

Shared semantic value variable used to pass attributes with tokens.

Semantic Values with %union

yylval can carry integers, strings, pointers or AST nodes.

```
%union {  
    int ival;  
    char *sval;  
}  
%token <ival> NUM  
%token <sval> ID  
%type <ival> expr
```

```
{DIGIT}+ { yyval.ival = atoi(yytext); return NUM; }  
{ID}     { yyval.sval = strdup(yytext); return ID; }
```

Teaching connection

Token name tells the parser the category. `yyval` tells it the value attached to this occurrence.

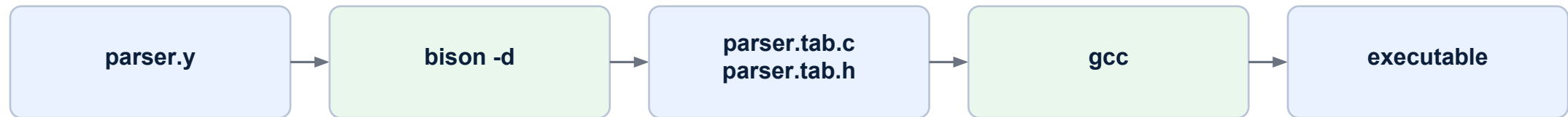
Build Commands: Flex + Bison

A simple compilation pipeline for lab work.

```
$ bison -d parser.y
$ flex scanner.l
$ gcc parser.tab.c lex.yy.c -lfl -o calc
$ ./calc
```

Order matters

Generate parser.tab.h before compiling scanner code that includes it.



Complete Mini-Scanner: Arithmetic

This scanner is enough for a calculator grammar.

```
%{
#include "parser.tab.h"
#include <stdlib.h>
%}
D [0-9]
%%
{D}+    { yylval.ival = atoi(yytext); return NUM; }
[
]+ ;
"+"|"-"|"*"|"/"|"("|")" return yytext[0];
.      { fprintf(stderr,"bad char: %s\n", yytext); }
%%
```

Tokens generated

NUM, +, -, *, /, (,)

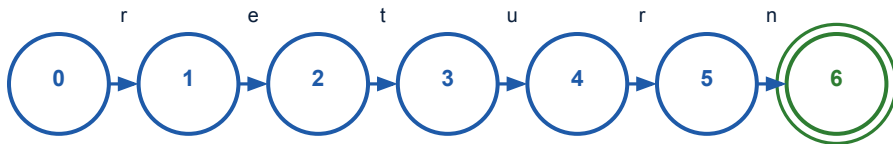
Lab task

Add variables: ID, assignment =, and symbol table lookup.

Keyword Table Alternative

Rather than writing many keyword rules, use a lookup table after ID match.

```
{ID} {  
  int tok = keyword_lookup(yytext);  
  if (tok) return tok;  
  yylval.sval = strdup(yytext);  
  return ID;  
}
```



Pros

Cleaner scanner when a language has many reserved words; keywords can be initialized in a table.

Cons

Slightly more runtime lookup; students must ensure exact keyword spellings and case rules.

Case Sensitivity and Keyword Design

Language design affects scanner rules.

Case-sensitive language

if is keyword
If may be ID
Simpler compiler convention
Common in C-like languages

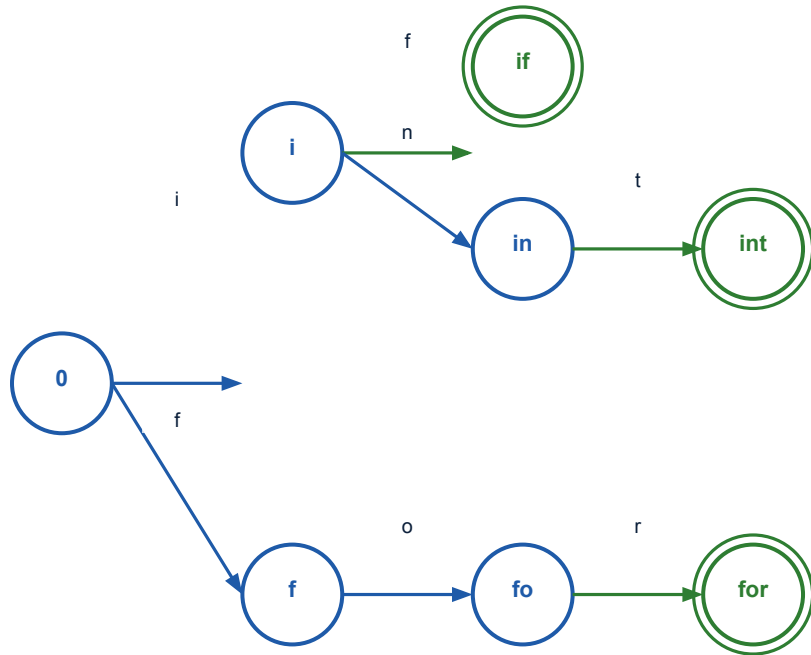
Case-insensitive language

IF, If, if same keyword
Need normalized lookup
Can use flex option or character classes
Common in SQL-like syntax

```
[iI][fF]    return IF;
```

Automata for Multiple Keywords

A combined trie-style DFA shares common prefixes.



Compiler intuition

Automata for keywords are not stored separately in generated scanners. They are combined and optimized.

Identifier conflict

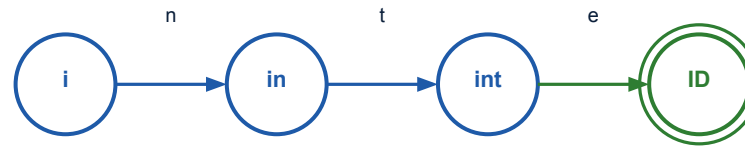
After accepting keyword if/int/for, a following letter or digit should continue into ID, not keyword.

DFA + Identifier Fallback

Keywords must not break identifiers such as integer or format.

```
"int"    return INT;  
"for"    return FOR;  
{ID}    return ID;
```

Input: integer



Why ID wins

Longest match consumes the full lexeme integer; keyword int is only a shorter prefix.

Scanner Performance Ideas

Generated scanners are designed to be fast.

Compile-time

Flex converts regex rules to NFA, then DFA-like tables and optimized code.

Run-time

The scanner advances through input using state transition tables; no backtracking is needed for common rules.

Optimization options

-Cf / -CF table compression choices
%option noinput nounput
Avoid REJECT in production scanners

Engineering trade-off

Readable rules first. Optimize only after correctness and diagnostics are stable.

Common Mistakes in Flex Labs

Most scanner bugs come from rule order, escaping and incomplete cases.

- Identifier rule before keyword rules
- Forgetting to skip whitespace
- Using . without error reporting
- Wrong escaping of *, +, \, "
- Not regenerating parser.tab.h
- Not assigning yylval before returning token

Quick diagnostic

Print each token as it is returned before connecting to yacc/bison. If the token stream is wrong, the parser cannot be right.

```
printf("TOKEN %d text=%s\n", tok, yytext);
```

Mini Case Study: if (x == 10)

Trace the scanner output before parsing.

Source: if (x == 10)

Lexeme	Token	Attribute
if	IF	—
(	(	—
x	ID	symbol pointer
==	EQ	—
10	NUM	10
)	)	—

Scanner delivers

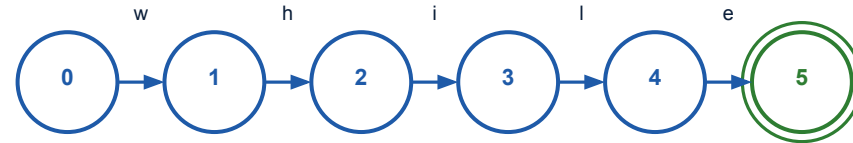
A clean token stream with attributes. Parser never sees spaces or raw characters inside identifiers/numbers.

Mini Case Study: while(count<=max)

Rules combine longest match and priority.

while(count<=max)

while	WHILE
(	(
count	ID
<=	LE
max	ID
)	)



Recognition points

while is keyword by priority; count and max are ID by general rule; <= is longer than <.

Lab Exercise 1: Token Printer

Build a scanner that prints token names for a small C-like language.

Required tokens

Keywords: if, else, while, int, return
Identifiers and integer literals
Operators: + - * / = == < <= > >= !=
Delimiters: () { } ; ,

Expected behaviour

Skip whitespace and comments
Report illegal characters with line number
Print one token per line
Show attributes for ID and NUM

```
Input:  int x = 10;  
Output: INT ID(x) '=' NUM(10) ';' ;
```

Lab Exercise 2: Connect to Yacc/Bison

Use the scanner as the front end for a small parser.



Parser grammar

expr, term, factor
assignment statements
if and while skeletons

Scanner requirements

Return exact token names
Fill yylval for ID and NUM
Track line numbers for yyerror()

Summary: What to Remember

Lex/Flex bridges compiler theory and practical front-end construction.

Core theory

Token = category; lexeme = text; pattern = regex; regular languages can be recognized by finite automata.

Core flex rules

Longest match first; if tie, earlier rule wins; put specific rules before general rules.

Core practice

Use start conditions for strings/comments, track line numbers, return token + yyval to yacc/bison.

Next topic

Syntax analysis: CFGs, parsing, yacc/bison grammar rules, parse conflicts and AST construction.

Recommended reference: Aho, Lam, Sethi & Ullman, Compilers: Principles, Techniques, and Tools — concepts paraphrased for teaching.